

Manual Tecnico

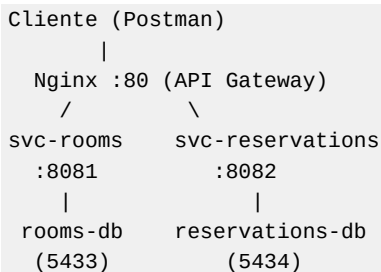
Sistema de Reservas Hotel Los Andes

Arquitectura de Microservicios - UTI

1. Introduccion

Este proyecto implementa la Actividad 3 de Microservicios: un sistema de reservas hoteleras para el Hotel Los Andes. Se basa en los patrones del proyecto comunicacionMicroServicios (DTOs, mappers, excepciones centralizadas) y en la infraestructura Docker del proyecto Biblioteca (docker-compose, nginx gateway, base de datos por servicio).

2. Arquitectura General



Reglas: cada microservicio tiene su propia base de datos PostgreSQL (Database per Service). La comunicacion entre svc-reservations y svc-rooms es exclusivamente HTTP REST. Todo el sistema se levanta con: docker-compose up -d --build

3. Estructura del Proyecto

```
Hotel/
|-- docker-compose.yml
|-- nginx/nginx.conf
|-- svc-rooms/
|   |-- pom.xml, Dockerfile
|   +-- src/main/java/com/uti/svcrooms/
|       |-- controller/ -> Endpoints REST
|       |-- service/impl/ -> Logica de negocio
|       |-- repository/ -> Acceso JPA
|       |-- model/ -> Entidades JPA
|       |-- dto/ -> Objetos de transferencia
|       |-- mapper/ -> Entity <-> DTO
|       +-- exception/ -> Manejo global de errores
|-- svc-reservations/
|   +-- src/main/java/com/uti/svcreservations/
|       |-- client/ -> RestTemplate y WebClient
|       |-- config/ -> Beans HTTP
|       +-- (misma estructura que svc-rooms)
+-- postman/Hotel_Los_Andes.postman_collection.json
```

4. Microservicio svc-rooms (Puerto 8081)

4.1 Entidad Room

Campos: id, roomNumber (unico), type (SINGLE/DOUBLE/SUITE), pricePerNight, totalCapacity, availableRooms, floor, description.

4.2 Endpoints

GET	/api/v1/rooms	-> Listar habitaciones
GET	/api/v1/rooms/{id}	-> Obtener por ID
POST	/api/v1/rooms	-> Registrar habitacion
GET	/api/v1/rooms/{id}/availability	-> Consultar disponibilidad
PATCH	/api/v1/rooms/{id}/availability	-> Actualizar disponibilidad

4.3 Reglas de negocio

- roomNumber duplicado -> 409 Conflict
- availableRooms negativo -> 422 Unprocessable Entity
- availableRooms > totalCapacity -> 422
- Errores con formato: timestamp, status, error, message, path

5. Microservicio svc-reservations (Puerto 8082)

5.1 Entidad Reservation

Campos: id, roomId, guestName, guestEmail, checkInDate, checkOutDate, status (ACTIVE/COMPLETED/CANCELLED), totalNights (calculado), createdAt.

5.2 Endpoints

```
POST /api/v1/reservations -> Crear reserva
GET /api/v1/reservations/{id} -> Obtener con datos del cuarto
GET /api/v1/reservations/guest/{email} -> Reservas por email
PATCH /api/v1/reservations/{id}/checkout -> Registrar salida
```

5.3 Comunicacion HTTP con svc-rooms

createReservation(): RestTemplate -> GET /availability; WebClient -> GET /rooms/{id}

getReservationById(): RestTemplate -> GET /rooms/{id}

getReservationsByEmail(): WebClient -> GET /rooms/{id}

checkout(): RestTemplate -> GET /rooms/{id}

5.4 RestTemplate vs WebClient

RestTemplate: cliente sincrono bloqueante. El hilo espera la respuesta HTTP. Ideal para operaciones secuenciales donde se necesita el resultado inmediato.

WebClient: cliente reactivo no bloqueante (Spring WebFlux). Permite programacion asincrona con Mono/Flux. En este proyecto se usa block() para integrarlo con servicios sincronos. La actividad exige usar ambos en metodos especificos.

6. Resilience4j - Patrones de Resiliencia

Configurado en svc-reservations para proteger llamadas a svc-rooms:

Circuit Breaker: sliding-window-size=5, failure-rate-threshold=50%, wait-duration=10s. Tras 3 fallos consecutivos el circuito se abre.

Retry: max-attempts=3, wait-duration=2s. Reintenta llamadas fallidas.

Fallback: cuando svc-rooms no responde:

- Crear reserva -> 503 'Room information temporarily unavailable'
- Consultar reserva -> 200 con roomNumber='Room information temporarily unavailable'

7. Nginx API Gateway

Nginx escucha en puerto 80 y enruta segun la URI:

- /api/v1/rooms* -> svc-rooms:8081
- /api/v1/reservations* -> svc-reservations:8082

El cliente (Postman) siempre apunta a http://localhost (puerto 80).

8. Docker Compose

Servicio	Puerto externo	Descripcion
nginx-gateway	80	API Gateway
svc-rooms	8081	Microservicio habitaciones

Hotel Los Andes - Sistema de Microservicios

svc-reservations	8082	Microservicio reservas
rooms-db	5433	PostgreSQL svc-rooms
reservations-db	5434	PostgreSQL svc-reservations

Comando de inicio: docker-compose up -d --build

9. Flujo de Creacion de Reserva

```
1. Cliente POST /api/v1/reservations
2. Nginx enruta a svc-reservations:8082
3. Validar fechas y reserva duplicada
4. RestTemplate -> svc-rooms GET /availability
5. Si available=false -> 422 'Room is not available'
6. WebClient -> svc-rooms GET /rooms/{id} (enriquecer respuesta)
7. Guardar reserva en reservations-db
8. RestTemplate PATCH availability (decrementar)
9. Retornar 201 con datos enriquecidos
```

10. Pruebas Minimas (Postman)

1. Registrar 3 habitaciones (SINGLE, DOUBLE, SUITE) -> 201
2. Crear reserva valida -> 201 enriquecida
3. Reserva sin disponibilidad -> 422
4. Reserva duplicada ACTIVE -> 422
5. Consultar reserva por ID -> 200 con datos del cuarto
6. Checkout reserva ACTIVE -> 200 COMPLETED
7. Checkout reserva COMPLETED -> 422
8. Parar svc-rooms, crear reserva -> 503 fallback
9. Consultar reserva con svc-rooms caido -> 200 fallback
10. Reiniciar svc-rooms -> recuperacion automatica

11. Concepto de Funcionamiento por Archivo

docker-compose.yml

Orquesta todos los contenedores: BDs, microservicios y nginx.

nginx/nginx.conf

Configura el API Gateway con reglas de enrutamiento por URI.

SvcRoomsApplication.java

Punto de entrada Spring Boot de svc-rooms.

Room.java

Entidad JPA mapeada a tabla rooms en PostgreSQL.

RoomController.java

Expone endpoints REST /api/v1/rooms.

RoomServiceImpl.java

Implementa reglas de negocio de habitaciones.

RoomMapper.java

Convierte entre entidad Room y DTOs sin exponer JPA.

GlobalExceptionHandler.java

Centraliza errores con formato uniforme JSON.

AppConfig.java

Hotel Los Andes - Sistema de Microservicios

Define beans RestTemplate y WebClient para svc-rooms.

RoomsRestTemplateClient.java

Cliente sincrónico con Circuit Breaker, Retry y Fallback.

RoomsWebClient.java

Cliente reactivo con mismos patrones de resiliencia.

ReservationServiceImpl.java

Orquesta reservas y llamadas HTTP a svc-rooms.

application.properties

Configura BD, URL de svc-rooms y Resilience4j.